

MSXPi: Bridging Generations — The Technical Odyssey of a Retro-Modern Hybrid

1. The Genesis of MSXPi: Motivation and Vision

The MSXPi project is not merely a peripheral; it is an architectural bridge across a forty-year chasm. Historically, 8-bit systems like the MSX—the Z80-powered standard of the 1980s—have existed in "hardware silos," isolated from the modern internet by the sheer physics of their era. The primary bottleneck was never the Z80's logic, but its inability to interact with modern storage and networking paradigms.

The vision of creator Ronivon Candido Costa was to resolve this "silo" problem by grafting the Raspberry Pi's ARM-based capabilities onto the MSX. This synthesis allows the Z80 to transcend the limitations of 720KB floppy disks and 3.58MHz processing. By utilizing the Pi as a co-processor and gateway, the MSXPi transforms a vintage computer into a functional modern terminal, capable of massive storage, WiFi connectivity, and real-time network services. This shifts the project's motivation from simple nostalgia to an exercise in functional expansion, proving that a forty-year-old system can still be a relevant node in a modern infrastructure.

2. System Architecture: The High-Level Blueprint

The MSXPi operates on a sophisticated Master-Server architecture designed to manage the clash of computing epochs between a 3.58MHz Z80 and a gigahertz-range ARM processor.

The Master-Server Model

In this dual-system logic, the MSX remains the **Client/Master**. It initiates all operations through custom BIOS calls or BASIC commands. The Raspberry Pi acts as the **Server**, a background appliance waiting to execute heavy lifting—such as HTTP requests, file decompression, or IRC management—that the Z80 cannot handle natively. This offloading strategy ensures the MSX remains the primary user interface while gaining the power of a modern Linux environment.

Data Flow Mapping

A typical command, such as `pdir` (remote directory listing), follows this logical path:

- **Command Initiation:** The user executes `CALL MSXPI("pdir")` at the BASIC prompt.
- **Buffer Capture:** The MSX client captures the command string from the system buffer at address `0x80`.
- **Logical I/O:** The command bytes are pushed through **DATA_PORT1 (0x5A)**, governed by the handshaking logic on **CONTROL_PORT1 (0x56)**.
- **Server Processing:** The Python-based server on the Pi interprets the `pdir` command, executes a Linux-side listing, and packages the result into 512-byte blocks.
- **Handshake/Return:** The server signals readiness; the MSX pulls the data blocks back through the data port, performing checksum verification on each block before printing to the screen.

3. Hardware Design: The Physical Interface

The hardware interface serves as the arbiter of synchronous reality, translating 5V Z80 timing into the 3.3V environment of the Raspberry Pi.

I/O Port Configuration and Environment Detection

The MSXPi occupies three critical ports in the MSX I/O map:

- **CONTROL_PORT1 (0x56):** The primary channel for handshaking and state control. It signals when the Raspberry Pi is ready for bit-level transmission.
- **CONTROL_PORT2 (0x57):** The "Environment Arbiter." In `msxpi-bios.c`, this port is polled to detect the system state. If the returned value is $< 0xFE$, the BIOS assumes physical MSXPi hardware; if it is $0xFE$ or higher, the system adapts to an emulated openMSX environment.
- **DATA_PORT1 (0x5A):** The 8-bit bidirectional data pipeline.

The Role of the CPLD

The physical hardware relies on a CPLD (Complex Programmable Logic Device) to act as an asynchronous-to-synchronous buffer. The CPLD manages the Z80's `WAIT` state and I/O request (`IORQ`) signals. When the Z80 attempts to read or write data, the CPLD can pause the CPU until the Raspberry Pi's GPIO pins have stabilized, ensuring that no data is lost during the bit-banging process.

Bit-Banging Mechanics

On the Raspberry Pi side, communication is handled via a Python-based SPI emulation over GPIO. Functions like `tick_sclk` and `SPI_ByteTransfer` manage the clock pulses and data bit alignment. The Pi must synchronize its high-speed cycles with the MSX by waiting for the `SPI_CS` and `RPI_READY` pins, effectively slowing the ARM down to the Z80's pace for the duration of the transfer.

4. The Communication Protocol: Handshaking and Reliability

Timing differences between a 1980s CPU and a modern SoC necessitate a custom Protocol v2, prioritizing reliability over raw speed.

Handshake and Parameter Transfer

Every transaction follows a strict sequence to ensure synchronization:

1. **MSX** sends `READY (0xAA)`.
2. **Pi** responds with `READY_ACK (0xA0)`.
3. **MSX** sends its maximum buffer size (`msxmaxbuf`) as two bytes in **Little-Endian** format (Low byte first, then High byte).
4. **Pi** configures its internal buffers to match.

Checksum Implementation

To minimize MSX overhead, a "Folded 16-bit checksum" is utilized. As implemented in `msxpi_bios.asm` (Lines 418-422), the system accumulates a 16-bit sum of all bytes in a 512-

byte block. This sum is then folded into an 8-bit value by adding the High byte to the Low byte and performing a logical AND `0xFF`.

Buffer Constraints

There is a vital discrepancy in buffer handling:

- The **Raspberry Pi Server** (`msxpi-server.py`) can handle buffers up to **48KB** (`48 * 1024`).
- The **MSX C-Client** (`msxpi.h`) is constrained to a `MAXBUFSIZE` of **8192 (8KB)**. While the protocol supports larger transfers, the 8KB limit in the MSX SDK is a strategic choice to accommodate the Z80's limited memory mapping and prevent stack collisions.

5. Software Design: The MSX Client-Side Logic

The MSX side provides a tiered software approach, offering BASIC accessibility while maintaining high performance through Assembly.

The BIOS and Assembly Foundation

The `msxpibios.asm` and `msxpi_bios.asm` files implement the `_MSXPI CALL` command. This layer handles the timing-sensitive port I/O and exposes routines like `PIREADBYTE` and `PIWRITEBYTE`. For larger transfers, the `RECVDATA` routine manages block-level reception and checksum verification.

Fusion-C and Memory Management

For developers, the project provides a C SDK via `msxpi-bios.c`. A critical safety feature is the `get_max_buffer_size` function. It prevents memory corruption by calculating the "safe zone" for data buffers: `SafeSize = (StackPointer - HeapTop - 32)`. This 32-byte safety margin ensures that large incoming data blocks from the Pi do not overwrite the MSX stack.

6. Software Design: The Raspberry Pi Server

The Raspberry Pi side is a Python 3 state machine that perpetually monitors the GPIO/Socket interface for commands.

Command Execution and Path Expansion

The server processes commands like `pcopy`, `pdir`, and `pdate`. A standout feature is the `pathExpander` logic in `msxpi-server.py`, which maps MSX-style drive letters to modern paths:

- **M:** maps to a networked cloud drive.
- **R1:** and **R2:** map to remote shares defined in `msxpi.ini`.
- The expander also supports direct URLs (`http/ftp`), allowing the MSX to "copy" files directly from the internet.

Appliance Integration

The `msxpi-monitor` script ensures the server behaves like a hardware appliance. It integrates with `systemd` to start at boot and manages a ramdisk at `/media/ramdisk` to store temporary files, minimizing SD card wear while accelerating file transfers.

7. Virtual openMSX Design and Implementation

The MSXPi includes a virtual driver for the openMSX emulator, allowing for development without physical hardware.

The MSXPiDevice Driver

In `MSXPiDevice.hh`, the emulator mimics the hardware ports (0x56-0x5A) by translating MSX IN/OUT instructions into TCP socket transmissions directed at the Python server's **Port 5000**.

Synchronization and Buffering

Because network sockets are inherently asynchronous compared to the deterministic timing of an emulator, the `MSXPiDevice` driver uses a `circular_buffer` (`rxQueue`) and `std::mutex` for synchronization. This ensures the emulated Z80 receives data with the same timing characteristics as the physical CPLD-based interface.

Feature	Physical MSXPi	Virtual openMSX
Communication Path	CPLD / GPIO Bit-banging	TCP Socket (Port 5000)
Port Logic (0x57)	Returns < 0xFE	Returns >= 0xFE
Synchronization	CPLD WAIT states	<code>std::mutex</code> / <code>rxQueue</code>
Logic Level	Physical 5V to 3.3V	Software Abstraction

8. Practical Implementation: Setup and Configuration

Deployment follows two paths: the "Easy Way" (pre-installed Pi image) or the "Hard Way" (manual Raspbian configuration).

The MSX-DOS Environment

MSXPi integrates with MSX-DOS via `msxdos_inihrd` and `dskiords`. By treating disk images on the Pi as virtual drives (A: and B:), the MSX gains a "virtual hard drive" with storage capacity far exceeding original hardware. Network configuration is simplified through the `pwifi` command and the `set` command (e.g., `SET DRIVEA /path/to/disk.dsk`), allowing the user to manage the Pi co-processor without leaving the MSX environment.

9. Conclusion: The Future and Legacy of MSXPi

The MSXPi project represents a significant evolution in retro-computing, moving from simple file transfer in v0.1 to a full BIOS-integrated subsystem in v1.4. It serves as a blueprint for other 8-bit

systems, proving that the gap between vintage logic and modern connectivity can be closed through thoughtful architecture. Future goals include deeper IRC integration and expanded network drive mappings, ensuring the MSX remains a functional, connected participant in the modern digital landscape.

Appendix: Command Reference & Technical Specs

MSX-Side Command Reference

Command	Arguments	Description
pdir	[path]	Lists files in current Pi directory or networked path (M:, R1:, etc.).
pcopy	[/z] [remote] [local]	Copies files. /z enables decompression of LZH/ROM files.
pdate	None	Synchronizes MSX system clock with the Raspberry Pi's Linux clock.
pwifi	[display / set]	Configures or displays the Raspberry Pi's WiFi connection status.
pplay	[cmd] [file]	Commands the Pi to play/stop/pause audio files via pplay.sh.
pvol	[0-100]	Adjusts the audio output volume on the Raspberry Pi.
pver	None	Displays the MSXPi Server version and Build ID.

Return Code Quick-Reference

Constant	Hex Value	Meaning
RC_SUCCESS	0xE0	Operation completed successfully.
RC_ESCPRESSED	0xE2	Operation aborted by user (ESC key).
RC_BUF0VFLW	0xE3	Data size exceeds MSX buffer capacity.
RC_HANDSHAKEERR	0xE5	Failure to establish sync between MSX and Pi.
RC_FILENOTFOUND	0xE6	Requested remote file could not be located.
RC_CONNERR	0xE8	General communication failure or socket error.

RC_CHKSUM_ERR	0xAD	Data corruption detected via folded 16-bit checksum.
RC_TERMINATE	0xED	Transfer session finalized and closed.